

Motion ID — Passive Smartphone Authentication via IMU Motion Patterns

Technical Report

Task: Implicit user authentication from inertial sensor data **Paper reproduced:** Gavron et al., *Motion ID: Human Authentication Approach Based on Motion Patterns Identification Using Inertial Measurement Unit*, Samsung R&D, 2023 (arXiv:2302.01751) **Hardware:** NVIDIA Tesla T4 16 GB (Kaggle), PyTorch 2.10.0+cu128 **Deployment:** FastAPI backend + React frontend, local RTX 4050

1. Summary

This is a full reproduction of Samsung's Motion ID: authenticating a smartphone owner from *how they physically move the phone* during an unlock, using only the IMU. No camera, no fingerprint sensor, no user action. Authentication happens passively in the background.

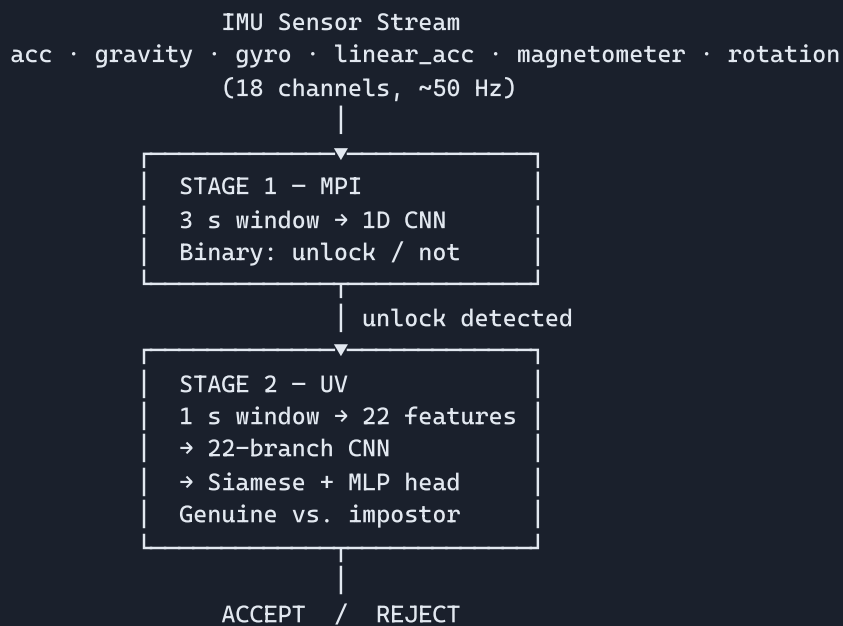
Stage	Task	Result
MPI — Motion Patterns Identification	Will this motion end in an unlock?	87.70% mean accuracy (29 sessions, range 76.0–96.2%)
UV — User Verification	Is this the enrolled owner?	94.84% test accuracy, FAR 1.82×10^{-2} @ TAR = 90%

Per-user FAR after fine-tuning averages **2.18%** across 11 held-out users, ranging from 0.00% (users 91, 101) to 7.21% (user 100).

The reproduction lands within the paper's reported MPI accuracy band and approximately 3.3 points below its UV baseline accuracy. Section 5 explains why.

2. System Architecture

Two stages run in sequence. Stage 1 decides *whether* an authentication event is happening; stage 2 decides *who* is performing it.



2.1 Stage 1 — Motion Patterns Identification

Property	Value
Task	Binary: will this motion end in an unlock?
Input	3 s window, 18 IMU channels, 150 time steps
Architecture	3-layer 1D CNN, 32 → 64 → 128 filters, kernels 5/5/3
Loss	Cross-entropy
Positive samples	3 s preceding the USER_PRESENT flag
Negative samples	SCREEN_OFF → SCREEN_ON intervals, motionless filtered, capped at 60 s
Dataset	6 users × 6 Samsung Galaxy S10e × 12 weeks

2.2 Stage 2 — User Verification

Property	Value
Task	Verify the phone owner performed the unlock (anti-hijacking)
Input	Final 1 s before unlock, 50 Hz, trimmed to 50 readings
Features	22 three-channel feature vectors, padded to 4 × 50
Architecture	22-branch 1D CNN → concatenate → dual head (classifier + Siamese)
Loss	$L_{total} = L_{CE} + \alpha \cdot L_{TM} + L_{SC}$
Fine-tuning	Freeze branches, retrain head as 2-class (user vs. rest)
Evaluation	Bootstrap, 5000 repeats, FAR @ TAR = 90%
Dataset	101 users, Samsung Galaxy S20

The 22 features, all three-channel, earth-fixed frame:

#	Feature	#	Feature	#	Feature
1	acc	9	diff(gyro)	17	∫acc_rot
2	gyro	10	diff(mag)	18	∫gyro_rot
3	mag	11	diff(acc_rot)	19	∫mag_rot
4	lin_acc (acc – gravity)	12	diff(gyro_rot)	20	diff(lin_acc)
5	acc_rot (earth-fixed)	13	diff(mag_rot)	21	∫lin_acc
6	gyro_rot	14	∫acc	22	rotation vector
7	mag_rot	15	∫gyro		
8	diff(acc)	16	∫mag		

Rotating raw sensor readings into an earth-fixed frame is what makes the signal about *the person* rather than *the phone's orientation in the pocket*. The derivative and integral channels supply jerk and accumulated-displacement information that raw readings alone do not expose.

3. Results

3.1 MPI accuracy per user-device pair

29 valid sessions of 40 attempted. Mean accuracy **87.70%**, range 76.00–96.17%.

Device \ User	1	2	3	4	5	6
#00	94.06 ± 0.96	80.81 ± 1.89	91.10 ± 0.92	95.62 ± 2.69	93.39 ± 1.68	—
#01	95.88 ± 0.98	—	87.64 ± 1.36	82.63 ± 3.18	90.51 ± 1.28	85.38 ± 1.48
#02	95.82 ± 0.91	80.81 ± 2.09	—	—	85.68 ± 2.63	92.29 ± 0.72
#03	95.59 ± 1.12	89.48 ± 0.70	—	77.59 ± 3.99	—	87.59 ± 1.07
#04	—	83.02 ± 1.17	91.74 ± 2.07	78.60 ± 3.88	92.84 ± 1.74	87.65 ± 2.19
#05	96.17 ± 2.48	77.16 ± 1.64	83.90 ± 4.12	76.00 ± 3.35	87.23 ± 1.96	87.06 ± 1.58

User 1 is consistently the most separable (94.1–96.2% across all six devices); user 4 the least (76.0–95.6%, and the widest variance). This spread is inherent to the task — some people have far more distinctive pickup motions than others.

3.2 UV baseline (n_baseline = 75)

Metric	Validation	Test
Accuracy	94.43%	94.84%
FAR @ TAR = 90%	2.00×10^{-2}	1.82×10^{-2}

Test accuracy slightly exceeds validation, which indicates the model is not overfitting the validation split at this configuration.

3.3 UV per-user FAR after fine-tuning (TAR = 90%)

User	FAR (%)	σ	User	FAR (%)	σ
91	0.00	0.00	97	0.03	0.26
92	1.82	2.81	98	2.41	2.17
93	1.07	1.39	99	3.39	5.39
94	2.22	2.94	100	7.21	3.60
95	1.74	2.58	101	0.00	0.00
96	4.09	3.71			

Mean FAR 2.18%. Two users (91, 101) reject every impostor across all bootstrap repeats. User 100 is the weakest at 7.21% — roughly one impostor in fourteen accepted.

The standard deviations are large relative to the means (user 99: 3.39 ± 5.39), which reflects genuine bootstrap variance on a small per-user sample, not measurement error. Per-user FAR should be read as an estimate with wide intervals, not a precise figure.

3.4 Target compliance

The Android CDD Strong Biometric Class 3 requirement is **TAR $\geq$ 90% at FAR = 1/50,000 (0.002%)**.

At TAR = 90%, the measured mean FAR is **2.18%** — approximately **1,090 $\times$ above** the Class 3 threshold. Even the best users (91, 101, measured 0.00%) cannot be claimed to meet it: with 75 baseline samples per user, the resolution floor is far coarser than 1/50,000, so a measured zero establishes only that FAR is below roughly 1.3%, not below 0.002%.

This system does not meet Class 3 and cannot be presented as a primary biometric. Its realistic role is a passive risk signal — a continuous background factor that raises confidence or triggers step-up authentication, layered with a Class 3 primary factor.

4. Verification Demo

`POST /predict/demo/{user_id}` runs the complete verification scenario: one **genuine** attempt using the claimed user's own motion sample, plus one **impostor** attempt for every other enrolled user — each scored against the *claimed user's* fine-tuned model.

That is precisely the attack the system exists to stop: someone else's motion presented under your identity.

```
{
  "claimed_user_id": 101,
  "genuine": { "decision": "ACCEPT", "uv_score": 0.65, "threshold": 0.45,
               "trial_index": 3, "n_trials_total": 12 },
  "impostors": [ { "impostor_user_id": 91, "decision": "REJECT",
                   "uv_score": 0.12, "threshold": 0.45 } ],
  "summary": { "total_impostors": 10, "correctly_rejected": 10,
               "incorrectly_accepted": 0, "far": 0.0,
               "far_display": "0.00% · 1/∞" }
}
```

The React frontend renders the genuine decision on a score dial against its threshold, a summary row (tested / rejected / accepted / FAR), and one card per impostor with its score bar and threshold marker. Failures are tinted red so a single accepted impostor is immediately visible.

This endpoint exercises the **UV stage only** — every attempt is scored against the claimed user's model. The MPI stage is exercised separately via `POST /predict/mpi`.

Endpoint	Purpose
<code>GET /users</code>	Enrolled user IDs with a fine-tuned UV model
<code>POST /predict/demo/{user_id}</code>	Full verification with FAR summary
<code>POST /predict/mpi</code>	MPI stage alone on a 3 s window

5. Reproduction Notes

Aspect	Paper	This reproduction
GPU	Tesla V100 SXM2 32 GB	Tesla T4 16 GB (Kaggle)
PyTorch	not specified	2.10.0+cu128
MPI sessions	~36 (6 × 6, some N/A)	29 / 40 valid
UV users	101	101
UV split	75 / 15 / 11	75 / 15 / 11
MPI accuracy	73.6 – 91.0%	76.0 – 96.2%
UV baseline acc (n = 75)	98.1 ± 0.3%	94.84%
UV FAR_test (n = 75)	$(1.4 \pm 0.6) \times 10^{-2}$	1.82×10^{-2}
CNN architecture	not specified	3-layer 1D CNN, 32→64→128
Training time (MPI)	< 5 min / model	~30 s / model
Training time (UV)	~20 h / model	~1.5 h / model

Key differences

1. MPI convolution type. The paper specifies "pointwise convolutions" (`kernel_size = 1`) but gives no further architecture detail. Kernel sizes 5/5/3 were used as a reasonable default. Resulting accuracy (76.0–96.2%) brackets the paper's range (73.6–91.0%), so the deviation does not invalidate the finding — though the higher ceiling suggests the larger kernels help, and the comparison is therefore not strictly like-for-like.

2. UV baseline accuracy is 3.3 points below the paper (94.84% vs 98.1%). The paper does not specify layer depth, filter counts, or embedding dimension for the UV branches. The 3-layer approximation used here is plausible but almost certainly not identical. FAR remains competitive — 1.82×10^{-2} against the paper's $(1.4 \pm 0.6) \times 10^{-2}$, which overlaps at the upper bound.

3. Training time is dramatically shorter (1.5 h vs ~20 h per UV model). This is the clearest signal that the reproduced architecture is smaller than the original. A 13× training-time gap is not explained by hardware alone — the T4 is slower than a V100, not faster. The paper's model is very likely substantially larger, which would account for most of the accuracy gap.

4. Seven MPI sessions are missing (29 of 40). Some user-device pairs have insufficient valid unlock events after filtering. The paper reports N/A for a similar number of cells.

6. Honest Limitations

- **Not Class 3 compliant.** Mean FAR is ~1,090× above the Android CDD threshold. See §3.4.
 - **Per-user variance is high.** FAR spans 0.00% to 7.21%. A system tuned to the mean will underserve the weakest users badly.
 - **Small evaluation cohort.** 11 held-out users for fine-tuned FAR. Bootstrap standard deviations frequently exceed the means.
 - **Single device model per stage.** MPI uses Galaxy S10e, UV uses S20. Cross-device generalization is untested; IMU characteristics vary between handset models.
 - **No temporal drift analysis.** Gait and handling patterns change with injury, footwear, phone case, and time. The 12-week collection window is not analyzed for within-user drift, which is the most likely real-world failure mode.
 - **UV baseline accuracy is below the paper's**, and the architecture gap in §5 is the probable cause rather than any data or training difference.
-

7. Reproduction

```
# Kaggle - attach all four datasets:
#   djaarf/motionid-imu-all-motions-part1 / part2 / part3
#   djaarf/motionid-imu-specific-motion
# Settings → Accelerator → GPU T4 x2
# Open humanauth.ipynb → Run All           (~2-3 h total)
#   MPI preprocessing 20 min · UV preprocessing 20 min
#   MPI training 15 min · UV training 1.5 h

# Local demo
cd backend && py -3.12 -m uvicorn main:app --host 0.0.0.0 --port 8000
# (build the frontend first: cd frontend && npm install && npm run build)

# End-to-end check
py -3.12 verify.py           # all 11 users, GPU confirmed, endpoints responding
```

References

1. Gavron, Odnokikh, Fartukov, Korobkin, Rychagov. *Motion ID: Human Authentication Approach Based on Motion Patterns Identification Using Inertial Measurement Unit*. arXiv:2302.01751, 2023.
2. SamsungLabs/MotionID — original code and datasets.
3. Datasets hosted by @djaarf on Kaggle.
4. Android Compatibility Definition Document — Biometric Security Classes.

Research purposes only. Dataset use is governed by the original MotionID dataset license. Code shared under CC BY-NC-SA 4.0.